



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

严谨 · 勤奋 · 求实 · 创新

数字系统设计

04-网络剪枝技术

贾弘洋 & 刘勇攀

2025/03/11





- **Why and How to Achieve Sparse NN Network**
- **Weight Sparsity Perspective**
 - **Unstructured Sparsity**
 - **Structural Sparsity**
 - **Unstructured v.s. Structured: Which is Better?**
 - **Frequency-domain Sparsity**
- **Activation Sparsity Perspective**
 - **Inter-Frame Sparsity**
 - **ROI-Based Sparsity**
- **Leveraging Sparsity in Storage**

/ Low-Power AI Demand



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University



Cloud AI

>1kW

~100W

~10W

<1W

Edge AI

Big data training

Big models

Fast response

Critical performance

~100 billions of devices^{*1}

- Low power
- Low latency
- Low cost

^{*1} : from Tractica.

/ Sparsity in Biological System by Pruning



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

- # synapses increases before 2 years old and, then decreases due to pruning, possibly to reduce resource (e.g., energy) usage

36 weeks
gestation



Newborn



3 months



6 months



2 years



4 years



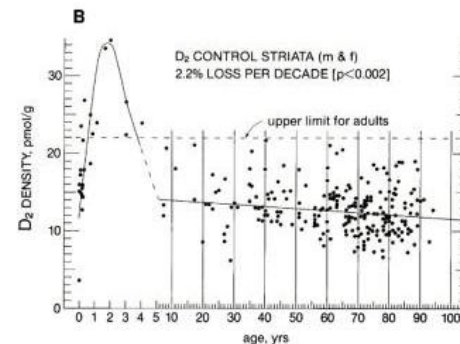
6 years



Synapse formation

Synapse pruning

Brain Development



Reproduced from Seeman et al.: Human Brain Dopamine Receptors in Children and Aging Adults. Synapse 1987; 1:399-404. Copyright © 1987, Wiley-Liss, Inc., a division of John Wiley and Sons, Inc. Reprinted by permission of John Wiley and Sons, Inc.

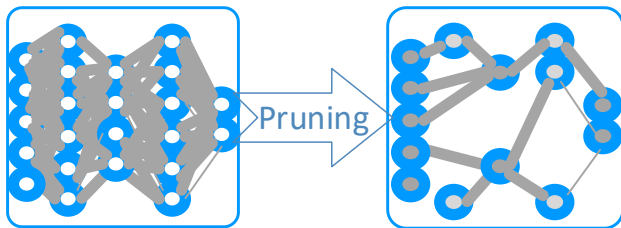
/ Sparsity: New Dimension For Efficiency



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

➤ Sparsity Origins:

- ❑ Pruning → Weights
- ❑ ReLU → Activations
- ❑ Domain specific.



9	-1	-3
1	-5	5
-2	6	-1



9	0	0
1	0	5
0	6	0

AlexNet Pruning Result	Method	Top 5 accuracy	Relative accuracy	Overall prun. rate
	SVD	79.4%	-0.9%	5.1x
	NeST	80.3%	+0.1%	15.7x
	ADMM-NN	80.3%	-1.9%	63.0x
	ADMM-NN	77.8%	-4.4%	96.0x

Optimize For **Sparsity** Can Significantly Improve Efficiency!

/ Source of Sparsity: Activation

- ReLU operator.

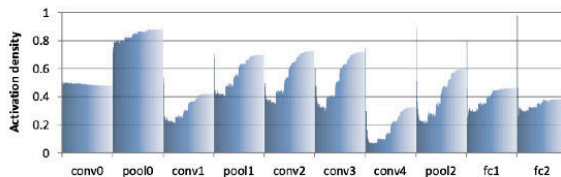
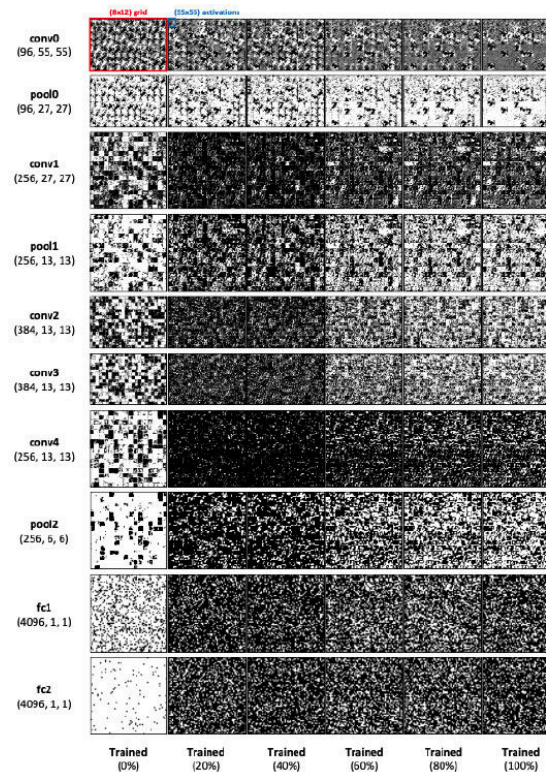


Fig. 4: Average activation density of each layer in AlexNet over time during training (going from dark to light blue colored bars, per layer). Activation density is sampled at every 2K iterations of training and a total of 226K iterations were spent to reach the fully trained model (53.1%/75.1% top-1/top-5 accuracy).

*Compressing DMA Engine:
Leveraging Activation Sparsity for
Training Deep Neural Networks,
HPCA'2018*



Source of Sparsity: Weights



- Pruning: if val. < threshold, val = 0;
- Regularization (Weight decay):
 - Expressing preferences for smaller weights
 - $CF = MSE_{train} + \lambda \sum_i w_i^2$ (L2)
 - $CF = MSE_{train} + \lambda \sum_i |w_i|$ (L1)

Table 4: For AlexNet, pruning reduces the number of weights by 9× and computation by 3×.

Layer	Weights	FLOP	Act%	Weights%	FLOP%
conv1	35K	211M	88%	84%	84%
conv2	307K	448M	52%	38%	33%
conv3	885K	299M	37%	35%	18%
conv4	663K	224M	40%	37%	14%
conv5	442K	150M	34%	37%	14%
fc1	38M	75M	36%	9%	3%
fc2	17M	34M	40%	9%	3%
fc3	4M	8M	100%	25%	10%
Total	61M	1.5B	54%	11%	30%

■ Remaining Parameters ■ Pruned Parameters

Layer	Remaining Parameters	Pruned Parameters
conv1	35	0
conv2	307	0
conv3	885	0
conv4	663	0
conv5	442	0
fc1	38	0
fc2	17	0
fc3	4	0
total	61	50

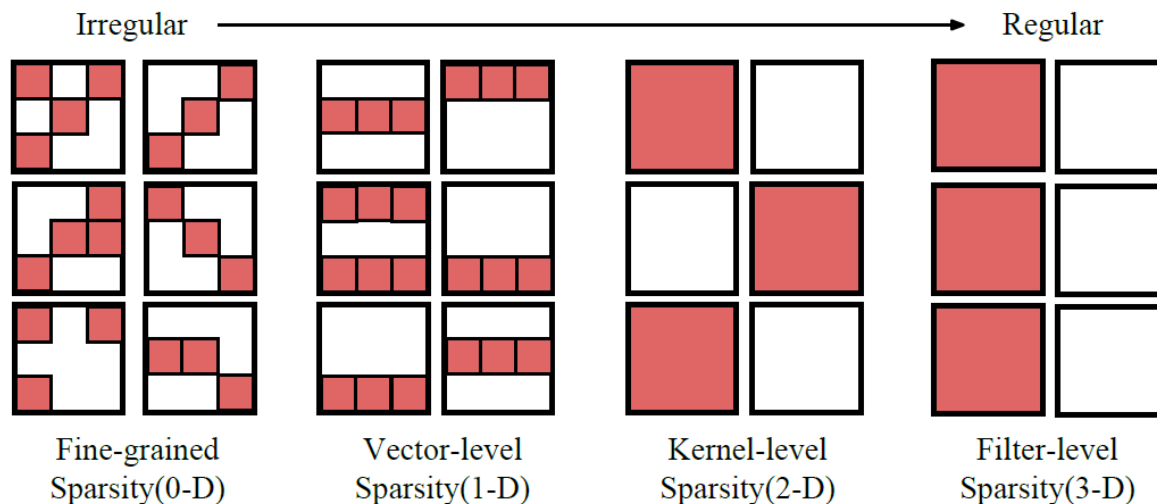


- **Why and How to Achieve Sparse NN Network**
- **Weight Sparsity Perspective**
 - **Unstructured Sparsity**
 - **Structural Sparsity**
 - **Unstructured v.s. Structured: Which is Better?**
 - **Frequency-domain Sparsity**
- **Activation Sparsity Perspective**
 - **Inter-Frame Sparsity**
 - **ROI-Based Sparsity**
- **Leveraging Sparsity in Storage**

Various Sparsity Granularity



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University



Compression ratio

Sparsity granularity

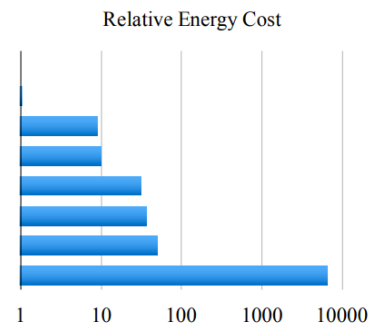
Source: CVPR'17 Workshop

/ Irregular Weight Pruning: Deep Compression



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

Operation	Energy [pJ]	Relative Cost
32 bit int ADD	0.1	1
32 bit float ADD	0.9	9
32 bit Register File	1	10
32 bit int MULT	3.1	31
32 bit float MULT	3.7	37
32 bit SRAM Cache	5	50
32 bit DRAM Memory	640	6400



Model	Size (M)	Top-1/top-5 error (%)	# layers	Model description
AlexNet	238	41.00/18.00	8	5 conv + 3 fc layers
VGG-16	540	28.07/9.33	16	13 conv + 3 fc layers
VGG-19	560	27.30/9.00	19	16 conv + 3 fc layers
GoogLeNet	40	29.81/10.04	22	21 conv + 1 fc layers
ResNet-50	100	22.85/6.71	50	49 conv + 1 fc layers
ResNet-152	235	21.43/3.57	152	151 conv + 1 fc layers

Memory access: 100x more energy than arithmetic.
@45nm CMOS technology (signal operation).

Large models for high accuracy:
need model size reduction!

Source: [Learning both Weights and Connections for Efficient Neural Network \(nips.cc\)](https://nips.cc/paper/learning-both-weights-and-connections-for-efficient-neural-network)

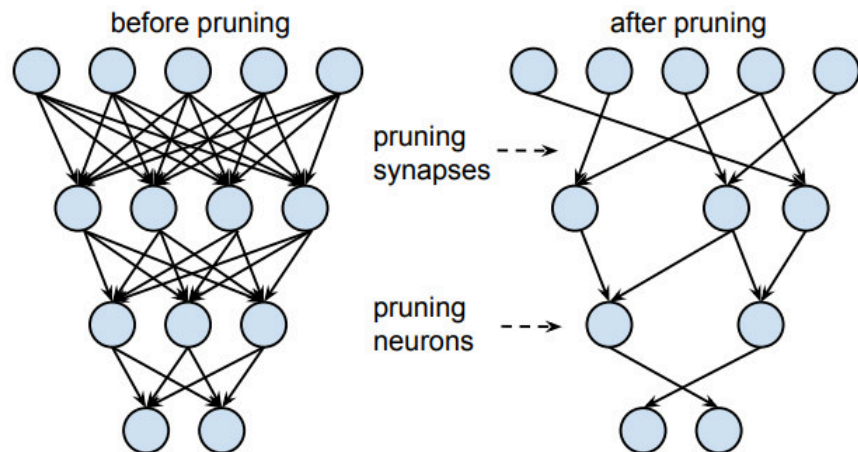
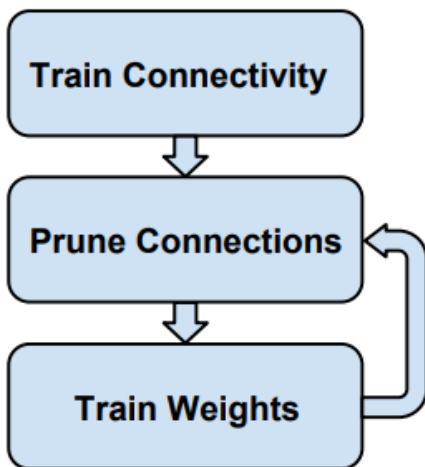
/ Irregular Weight Pruning: Deep Compression



Three step strategy:

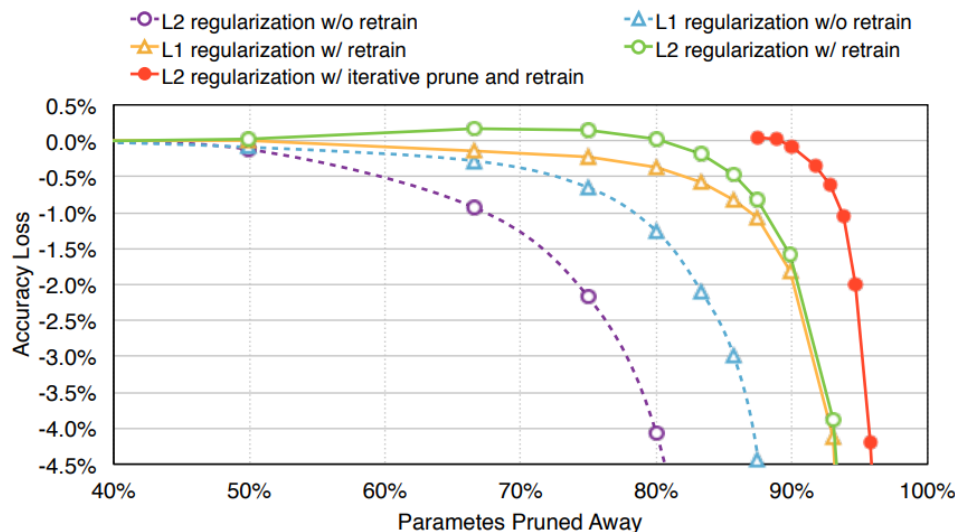
1. Train network to full accuracy.
2. Pruning weights whose absolute value are under a threshold.
3. If the accuracy satisfies the requirement, then end.

Else retrain the network and go to 2.



Source: [Learning both Weights and Connections for Efficient Neural Network \(nips.cc\)](https://arxiv.org/abs/1506.02558)

Irregular Weight Pruning: Deep Compression



Key for this approach:

1. regularization method. (L2 here, better methods in latter approaches).
2. iteratively prune and retrain (time/energy consuming). Traditionally pruning needed to be off-line for retraining.

Source: [Learning both Weights and Connections for Efficient Neural Network \(nips.cc\)](#)

/ Irregular Weight Pruning: Deep Compression



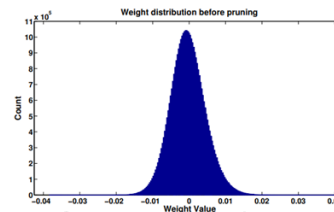
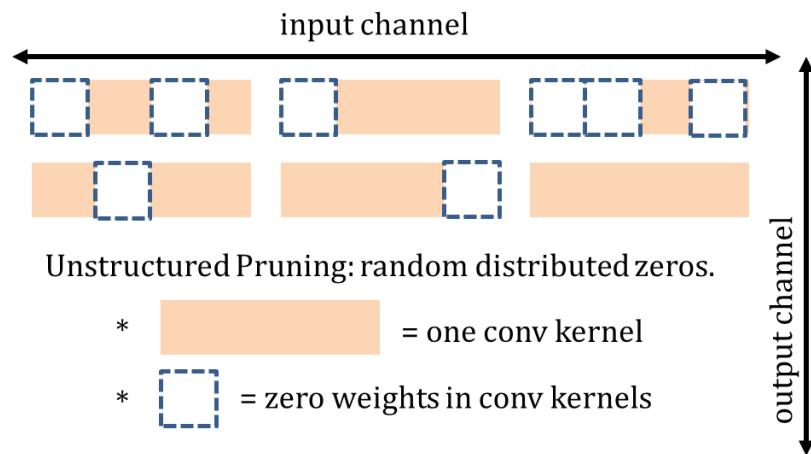
清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

Pruning weights with small absolute value: **Irregular Pruning**.

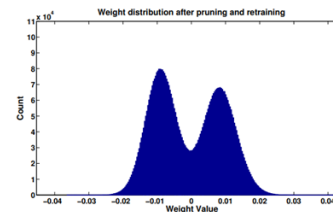
Results: Randomly distributed zero weights.

Pros: **High compression ratio** w/o accuracy loss.

Cons: **Inefficient for hardware acceleration, high storage overhead.**



Before Pruning (10x Y-axis)



After Pruning

Network	Original Size	Compressed Size	Compression Ratio	Original Accuracy	Compressed Accuracy
AlexNet	240MB	→ 6.9MB	35x	80.27%	→ 80.30%
VGGNet	550MB	→ 11.3MB	49x	88.68%	→ 89.09%
GoogLeNet	28MB	→ 2.8MB	10x	88.90%	→ 88.92%
SqueezeNet	4.8MB	→ 0.47MB	10x	80.32%	→ 80.35%

Source: [Learning both Weights and Connections for Efficient Neural Network \(nips.cc\)](https://nips.cc/paper/learning-both-weights-and-connections-for-efficient-neural-network)

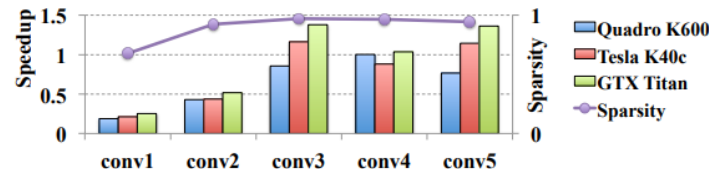


- **Why and How to Achieve Sparse NN Network**
- **Weight Sparsity Perspective**
 - **Unstructured Sparsity**
 - **Structural Sparsity**
 - **Unstructured v.s. Structured: Which is Better?**
 - **Frequency-domain Sparsity**
- **Activation Sparsity Perspective**
 - **Inter-Frame Sparsity**
 - **ROI-Based Sparsity**
- **Leveraging Sparsity in Storage**

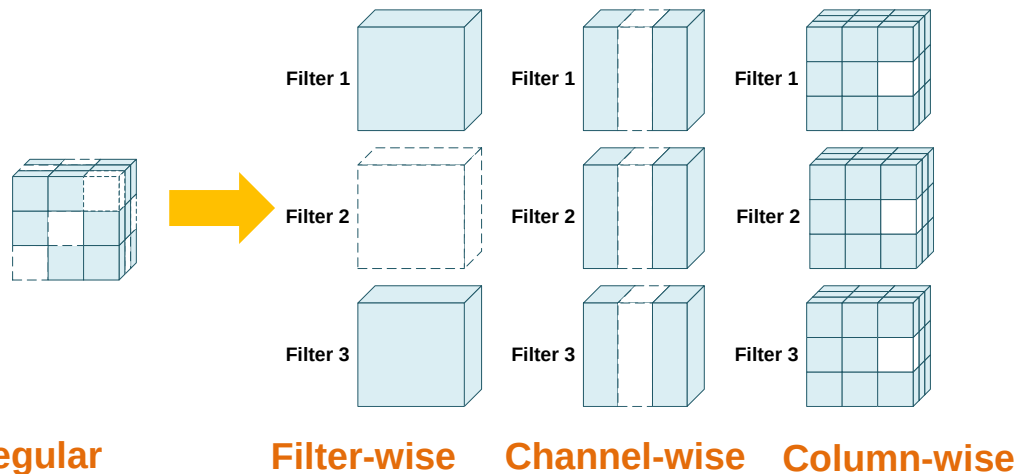
Structured Weight Pruning: SSL



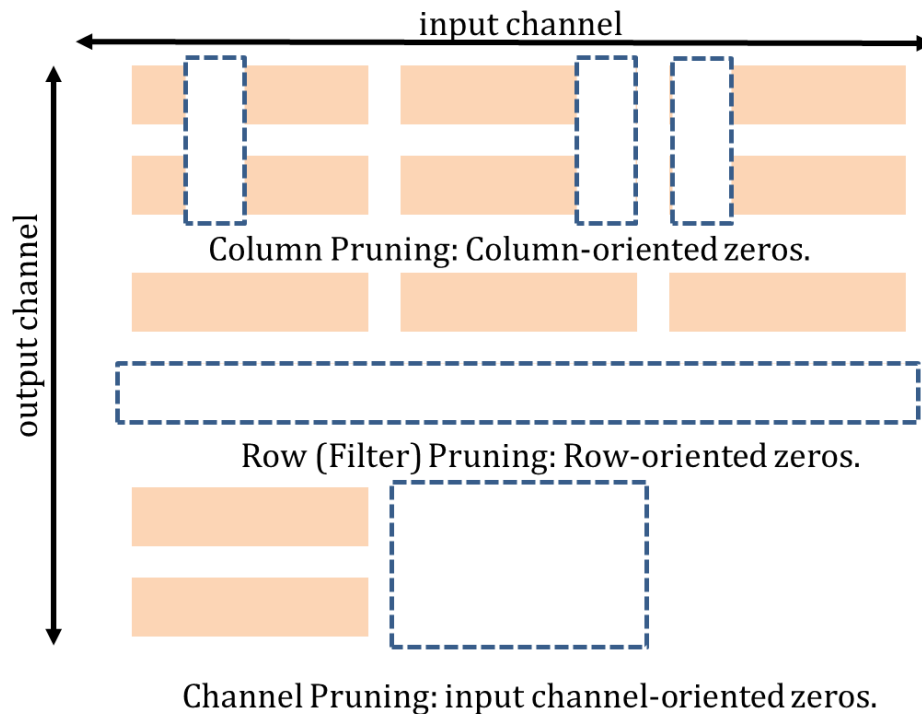
- Irregular weight pruning:
 - HW speedup does not match pruning rate for irregularity.
 - Dedicated device is required!!



- SSL: structured weight pruning



/ Structured Weight Pruning: SSL



/ Structured Weight Pruning: SSL



- SSL method

- Group-Lasso regularization.

- g: represents a group (of weights).
 - G: represents the number of groups.
 - Different groups may overlap.

$$E(\mathbf{W}) = E_D(\mathbf{W}) + \lambda \cdot R(\mathbf{W}) + \lambda_g \cdot \sum_{l=1}^L R_g \left(\mathbf{W}^{(l)} \right).$$

$$R_g(\mathbf{w}) = \sum_{g=1}^G \|\mathbf{w}^{(g)}\|_g$$

$$\|\mathbf{w}^{(g)}\|_g = \sqrt{\sum_{i=1}^{|\mathbf{w}^{(g)}|} \left(w_i^{(g)} \right)^2}$$

/ Structured Weight Pruning: SSL



- SSL method
 - Group-Lasso regularization.
- Different structure: different method of splitting groups of weights.
- Retraining technique is used.

$$E(\mathbf{W}) = \underbrace{E_D(\mathbf{W})}_{\text{Acc. loss}} + \underbrace{\lambda \cdot R(\mathbf{W})}_{\text{non-structural}} + \underbrace{\lambda_g \cdot \sum_{l=1}^L R_g(\mathbf{W}^{(l)})}_{\text{structural (group-lasso)}}.$$

$$E(\mathbf{W}) = E_D(\mathbf{W}) + \underbrace{\lambda_n \cdot \sum_{l=1}^L \left(\sum_{n_l=1}^{N_l} \|\mathbf{W}_{n_l, :, :, :}^{(l)}\|_g \right)}_{\text{filter structure}} + \underbrace{\lambda_c \cdot \sum_{l=1}^L \left(\sum_{c_l=1}^{C_l} \|\mathbf{W}_{:, c_l, :, :}^{(l)}\|_g \right)}_{\text{channel structure}}.$$

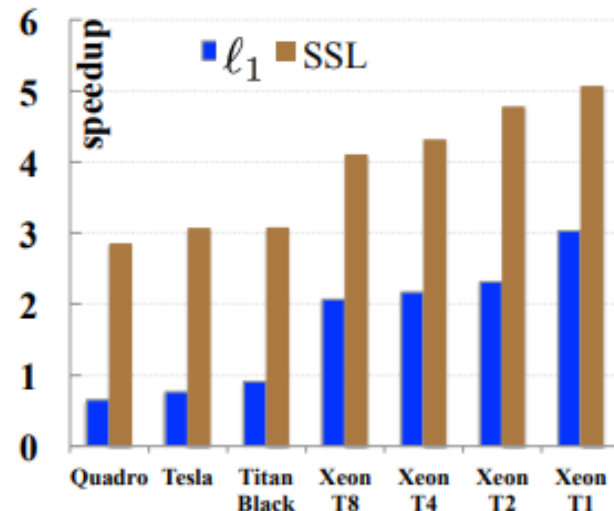
$$E(\mathbf{W}) = E_D(\mathbf{W}) + \underbrace{\lambda_s \cdot \sum_{l=1}^L \left(\sum_{c_l=1}^{C_l} \sum_{m_l=1}^{M_l} \sum_{k_l=1}^{K_l} \|\mathbf{W}_{:, c_l, m_l, k_l}^{(l)}\|_g \right)}_{\text{column (shape) structure}}.$$

Structured Weight Pruning: SSL

- Structured pruning:
 - Significant speedup on general devices

#	Method	Top1 err.	Statistics	conv1	conv2	conv3	conv4	conv5
1	ℓ_1	44.67%	sparsity	67.6%	92.4%	97.2%	96.6%	94.3%
			CPU $\times$	0.80	2.91	4.84	3.83	2.76
			GPU $\times$	0.25	0.52	1.38	1.04	1.36
2	SSL	44.66%	column sparsity	0.0%	63.2%	76.9%	84.7%	80.7%
			row sparsity	9.4%	12.9%	40.6%	46.9%	0.0%
			CPU $\times$	1.05	3.37	6.27	9.73	4.93
			GPU $\times$	1.00	2.37	4.94	4.03	3.05
3	pruning [7]	42.80%	sparsity	16.0%	62.0%	65.0%	63.0%	63.0%
4	ℓ_1	42.51%	sparsity	14.7%	76.2%	85.3%	81.5%	76.3%
			CPU $\times$	0.34	0.99	1.30	1.10	0.93
			GPU $\times$	0.08	0.17	0.42	0.30	0.32
5	SSL	42.53%	column sparsity	0.00%	20.9%	39.7%	39.7%	24.6%
			CPU $\times$	1.00	1.27	1.64	1.68	1.32
			GPU $\times$	1.00	1.25	1.63	1.72	1.36

Accuracy and Sparsity of AlexNet on ImageNet



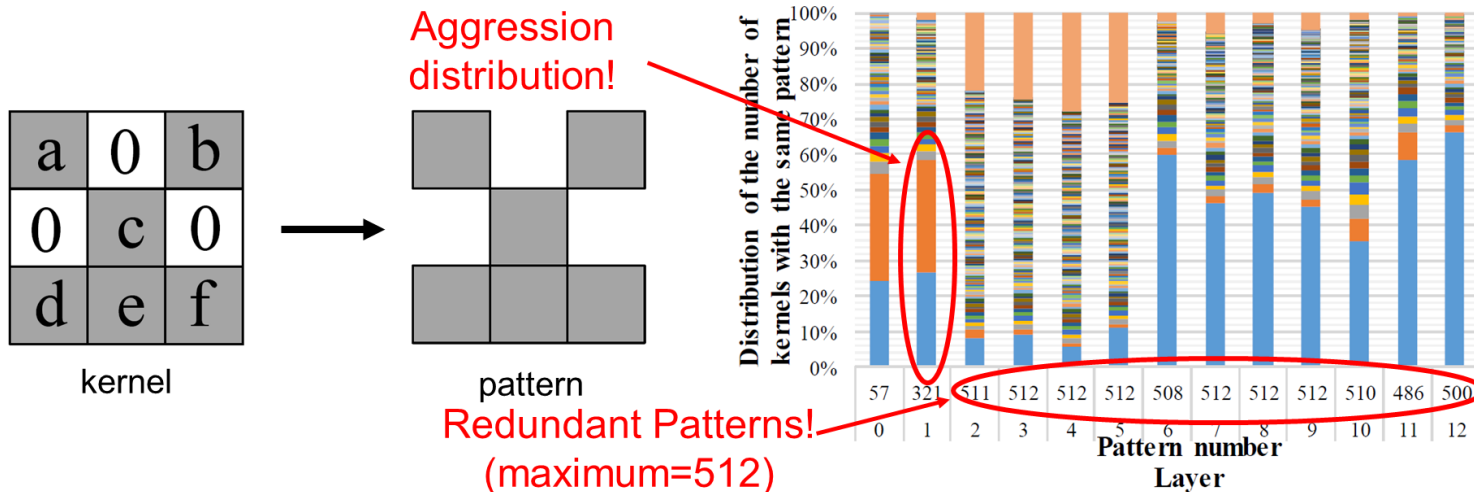
Speedup on AlexNet

Source: [Learning Structured Sparsity in Deep Neural Networks \(neurips.cc\)](https://arxiv.org/abs/1507.02646)

Emerging Structured: Pattern Pruning



- Redundant Patterns of Conv Kernels



Pattern: indices of nonzero weights in a conv kernel.

Pattern number and distribution in VGG16.

How can we attain a network with limited patterns while keeping reasonable accuracy and sparsity?

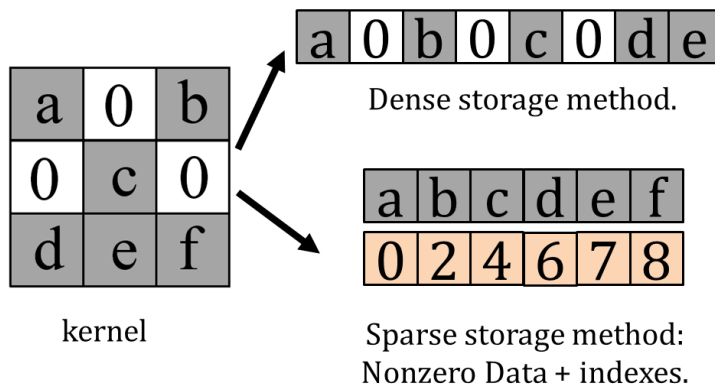
/ Emerging Structured: Pattern Pruning

- Lack of Intra-Kernel Structured Pruning.

Pruning Scheme	Accuracy				Hardware Speedup			
	Highest	Minor Loss	Moderate Loss	Highest Loss	Highest	High	Moderate	Minor
Non-structured	X							X
Filter/Channel				X	X			
Pattern	X				X			
Connectivity		X				X		

Emerging Structured: Pattern Pruning

- Index Storage Matters for Sparse NNs.

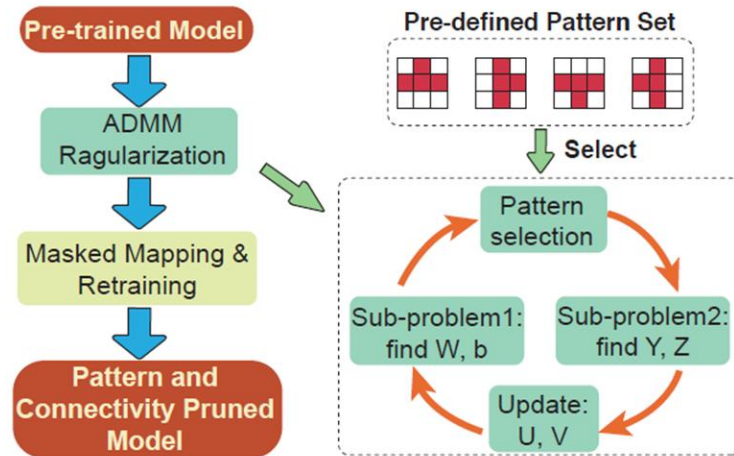
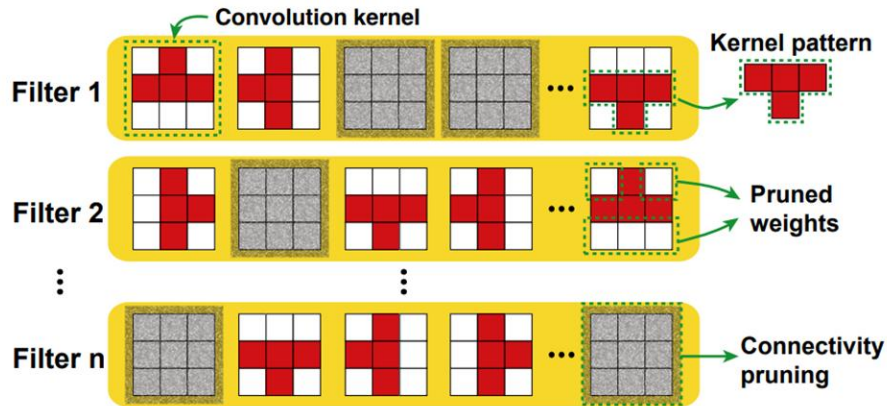


Dense and Sparse storage methods for kernels.

For 4bit weight:
index storage cost = data storage cost.

If we manage to greatly reduce pattern numbers, what storage scheme is efficient for these pattern-compressed networks?

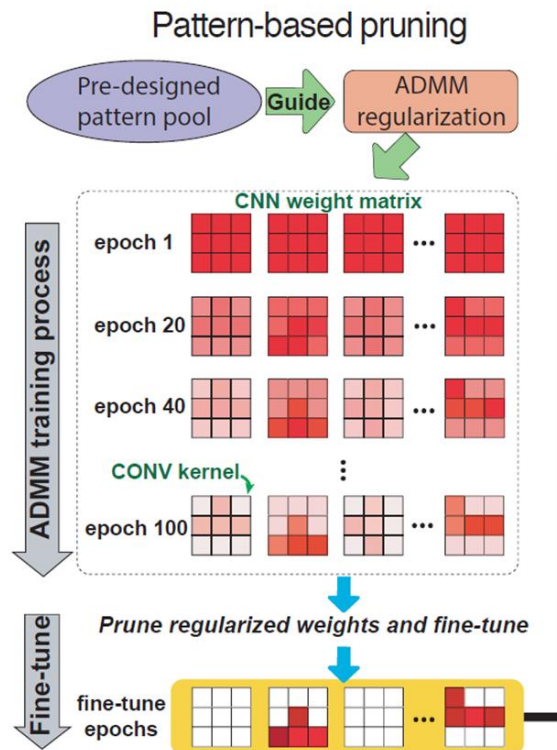
/ Fixed-Length Pattern Pruning



Fixed number of weights are pruned in a convolution kernel (4 nonzero entries in a nonzero kernel).

Source: [PCONV: The Missing but Desirable Sparsity in DNN Weight Pruning for Real-Time Execution on Mobile Devices](#)

/ Fixed-Length Pattern Pruning



Step1: Pre-design the 4-entry pattern pools.

Step2: Train the network while regularizing the weights to fit in the pre-designed patterns.

Step3: Pruning and Fine-tuning.

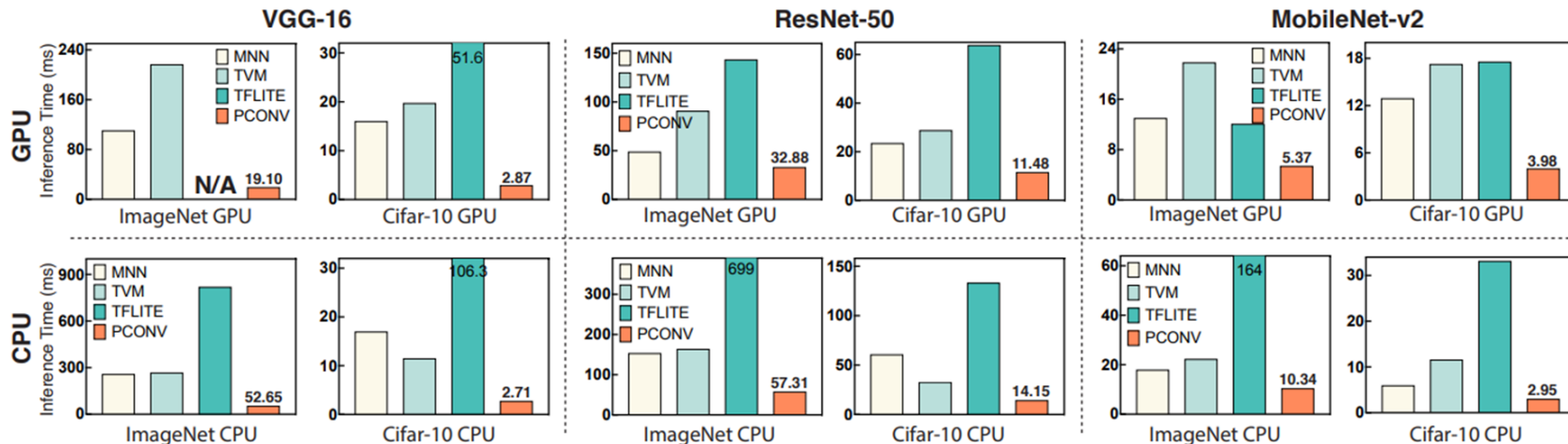
Result: All kernels projected to pre-designed pattern set with 4 nonzero entries.

VGG@CIFAR-10: 8 pattern, -0.2% accuracy.

/ Fixed-Length Pattern Pruning

Pros: Appealing for hardware acceleration. (CPU/GPU/FPGA/ASIC).

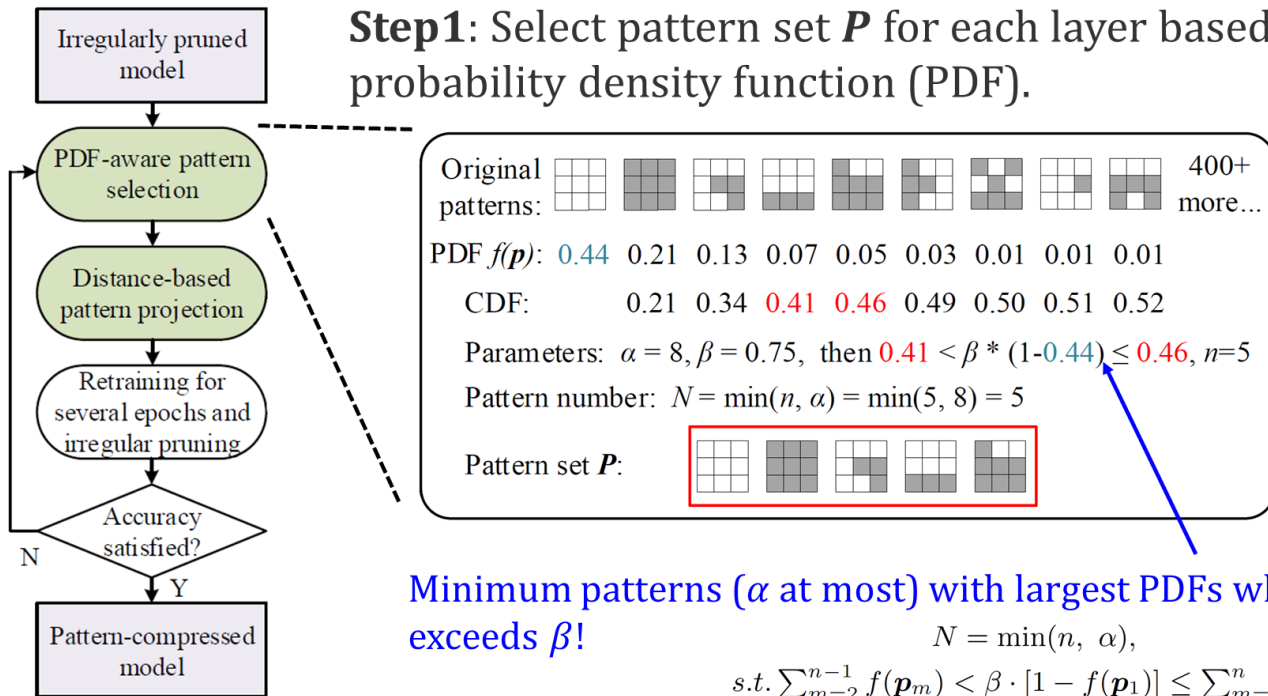
Cons: Less compression rate (compared with irregular pruning).



Inference time (ms) showcase.

Source: [PCONV: The Missing but Desirable Sparsity in DNN Weight Pruning for Real-Time Execution on Mobile Devices](#)

Flexible-Length Pattern Pruning



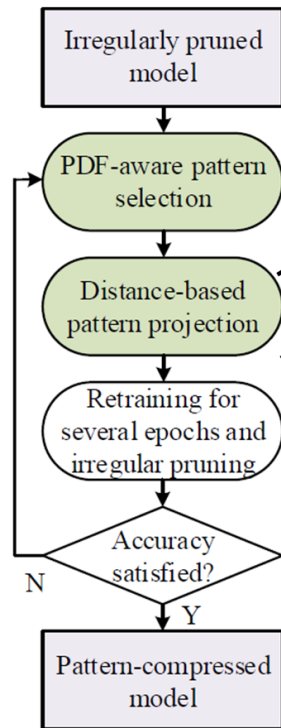
Minimum patterns (α at most) with largest PDFs whose sum exceeds β !

$$N = \min(n, \alpha),$$

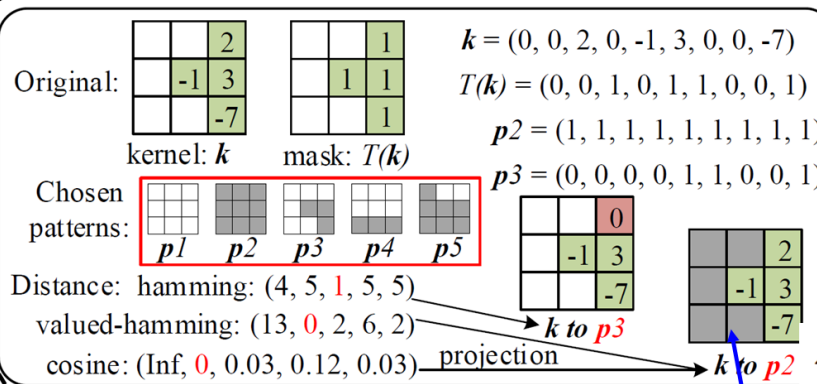
$$s.t. \sum_{m=2}^{n-1} f(p_m) < \beta \cdot [1 - f(p_1)] \leq \sum_{m=2}^n f(p_m)$$

$$P = \{p_m \mid 1 \leq m \leq N\}$$

Flexible-Length Pattern Pruning



Step2: Project kernels to the target weight set $\mathcal{S}_i^P = \{W_i | \text{card}(W_i) \leq l_i, T(W_i^{m,n}) \in P_i\}$ based on the distances to patterns (multiple distances to choose).



$$Z_i^{k+1} = \Pi_{\mathcal{S}_i}(U_i^k + W_i^{k+1})$$

$$U_i^{k+1} = U_i^k + W_i^{k+1} - Z_i^{k+1}$$

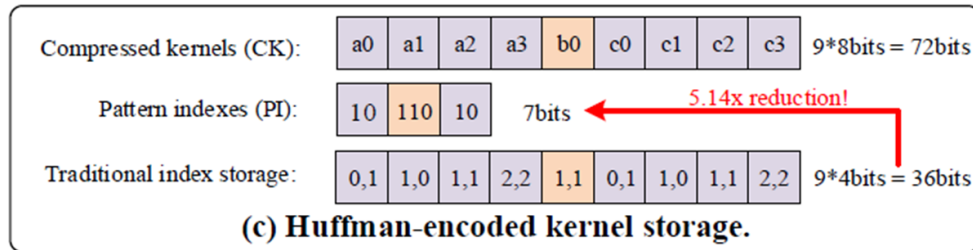
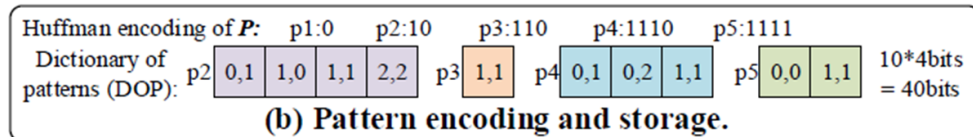
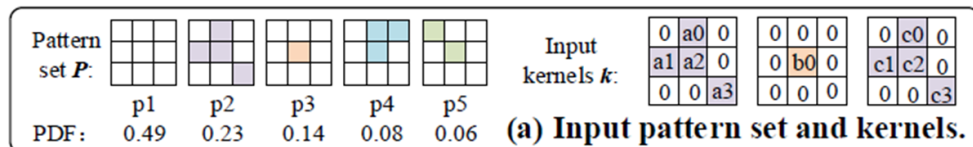
$$D_h(k, p) = \|p - T(k)\|_1$$

$$D_h(k, p) = |p - T(k)| \cdot |k|$$

$$D_c(k, p) = 1 - \frac{k \cdot (k \otimes p)}{\|k\|_2 \cdot \|k \otimes p\|_2}$$

Result: All kernels projected to $\mathcal{S}_i^P$ with min distance!
 Using ADMM method to regularize weight to pattern.
 ADMM: Check Zhang TY@ECCV2018

Flexible-Length Pattern Pruning



Traditional storage: compressed kernels (CK) + corresponding indexes.

Our storage: compressed kernels (CK) + pattern indexes of kernels (PI) + dictionary of patterns (DOP).

DOP is negligible because it is shared by all kernels in one conv layer.

Result: index storage is reduced by 5.14x!

$$S(\text{PI}) = -\sum_{m=1}^N f(p_m) \log_2 f(p_m)$$

$$S(\text{DOP}) = (\lceil \log_2(W) \rceil + \lceil \log_2(H) \rceil) \cdot \sum_{m=1}^N \|p_i\|_1$$

/ Flexible-Length Pattern Pruning

Dataset	Distance function	α	β	Sparsity	Pattern number avg / min / max	Top-1 accuracy	Top-5 accuracy	Index storage reduction
CIFAR-10	D_h	Inf	0.4	85.15% (+3.20%)	4.23 / 2 / 9	91.47% (-1.25%)	/	5.51x
	$D_{\tilde{h}}$	10	0.3	82.48% (+0.53%)	3.85 / 2 / 7	91.85% (-0.87%)	/	5.63x
	D_c	15	0.3	80.67% (-1.28%)	4.62 / 2 / 13	92.88% (+0.16%)	/	6.27x
CIFAR-100	D_c	9	Inf	84.08% (+2.13%)	5.62 / 2 / 9	71.42% (-1.30%)	91.74% (+0.30%)	2.87x
	$D_{\tilde{h}}$	16	0.5	82.01% (+0.06%)	5.84 / 2 / 12	71.86% (-0.86%)	91.62% (+0.18%)	3.54x
	D_c	10	Inf	80.57% (-1.38%)	5.69 / 2 / 10	72.37% (-0.35%)	91.89% (+0.45%)	3.47x

*data tested on VGG16 network.

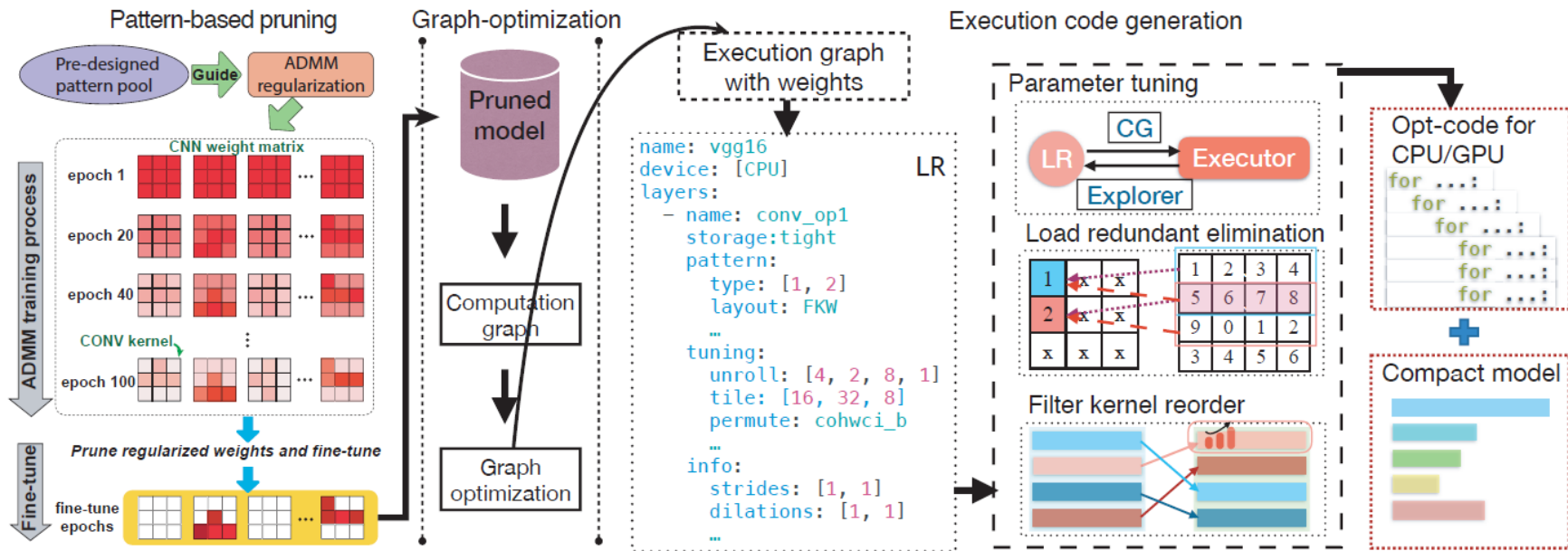
CIFAR-10: **5.63x**(1.38x) index(total) reduction, **3.85** average patterns, -0.87% top-1 accuracy, +0.53% sparsity.

CIFAR-100: **3.47x**(1.31x) index(total) reduction, **5.69** average patterns, -0.35%(+0.45%) top-1(top-5) accuracy, -1.38% sparsity.

/ Pattern Pruning + Compilation: CoCoPIE



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

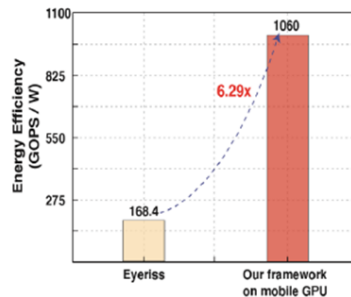
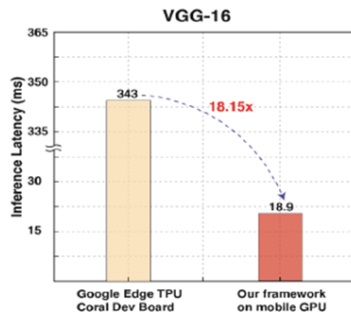
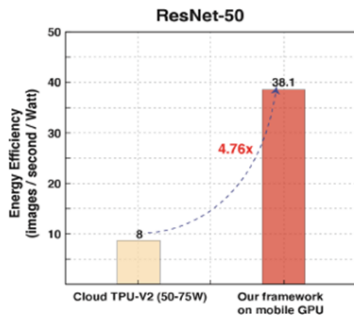


Source: [2003.06700.pdf \(arxiv.org\)](https://arxiv.org/pdf/2003.06700.pdf)

/ Pattern Pruning + Compilation: CoCoPIE

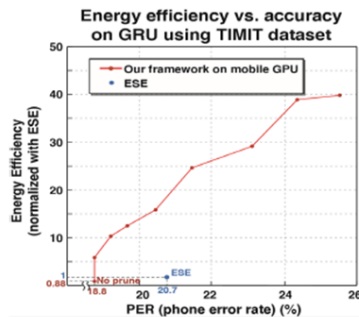
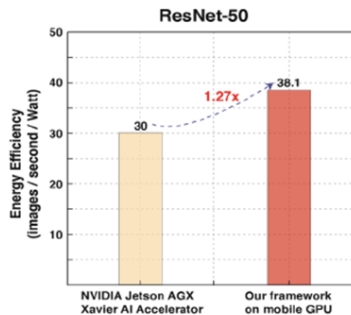
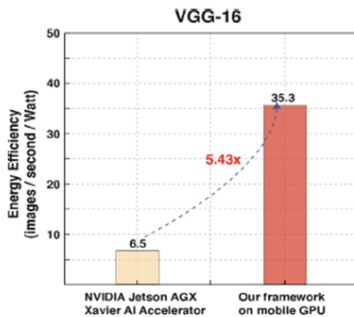


清华大学电子工程系
Department of Electronic Engineering, Tsinghua University



(a) Google TPU

(b) Eyeriss



(c) NVIDIA Xavier

(d) ESE

Energy Efficiency showcase (Images/sec/Watt) on FPGA & ASIC.



- **Why and How to Achieve Sparse NN Network**
- **Weight Sparsity Perspective**
 - **Unstructured Sparsity**
 - **Structural Sparsity**
 - **Unstructured v.s. Structured: Which is Better?**
 - **Frequency-domain Sparsity**
- **Activation Sparsity Perspective**
 - **Inter-Frame Sparsity**
 - **ROI-Based Sparsity**
- **Leveraging Sparsity in Storage**

/ Unstructured v.s. Structured



- Non-structured pruning
 - **Pros:** Higher compression rate
 - **Cons:** Only beneficial on dedicated devices
- Structured pruning:
 - **Pros:** Better suitable for more hardware devices
 - **Cons:** Lower compression rate

Could structure and non-structure pruning have similar compression ratio?

	GPU	Multi-core CPU	FPGA	ASIC
Non-Structured Pruning	Not Suitable	Not Suitable	Storage/Computation Dependent ?	
Structured Pruning	Suitable	Suitable		

Is non-structured pruning beneficial at all?

Source: [Non-Structured DNN Weight Pruning—Is It Beneficial in Any Platform?](#)

/ Unstructured v.s. Structured



- How to compare Structured vs Non-Structured Pruning?
 1. Compare with the *same accuracy and SoA practice*.
 2. Compare storage: *weight + indices (absolute/relative)*
 3. Compare *Pruning-to-Performance Ratio (PPR)*.
 - α x weight reduction, β x speedup, $PPR = \alpha/\beta$
 - less PPR $\rightarrow$ higher computation efficiency
- For (some) structured pruning, **PPR=1**
- SoA non-structured pruning, **PPR>2.7**
- Conclusion: non-structured pruning is more beneficial if it can achieve **2.7x more pruning rate** than structured pruning.



- **Why and How to Achieve Sparse NN Network**
- **Weight Sparsity Perspective**
 - **Unstructured Sparsity**
 - **Structural Sparsity**
 - **Unstructured v.s. Structured: Which is Better?**
 - **Frequency-domain Sparsity**
- **Activation Sparsity Perspective**
 - **Inter-Frame Sparsity**
 - **ROI-Based Sparsity**
- **Leveraging Sparsity in Storage**

Frequency Domain Sparsity: CirCNN

-1.39	0.06	1.56	0.36
0.72	-0.41	0.54	0.15
0.32	1.21	2.43	0.52
1.71	-3.23	-1.22	0.82

 $\times$

0.36
0.43
-0.12
3.42

 $=$

0.57
0.53
2.12
2.18

-1.39	0.06	1.56	0.36
0.36	-1.39	0.06	1.56
1.56	0.36	-1.39	0.06
0.06	1.56	0.36	-1.39

 $\times$

0.36
0.43
-0.12
3.42

 $=$

0.57
4.86
1.09
-4.01

Unstructured Weight

Storage: $O(n^2)$



Block-Circulant Weight
Conv kernel is represented by **vector**.

Storage: $O(n)$

* A theoretical proof of the effectiveness of cirCNN was also proposed.

/ Frequency Domain Sparsity: CirCNN



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

Ding CW@MICRO 2017

0.36	-1.39	0.06	0.43	-0.24	3.42	-0.12	1.56
1.56	0.36	-1.39	0.06	0.43	-0.24	3.42	-0.12
-0.12	1.56	0.36	-1.39	0.06	0.43	-0.24	3.42
3.42	-0.12	1.56	0.36	-1.39	0.06	0.43	-0.24
...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...

(a) Baseline

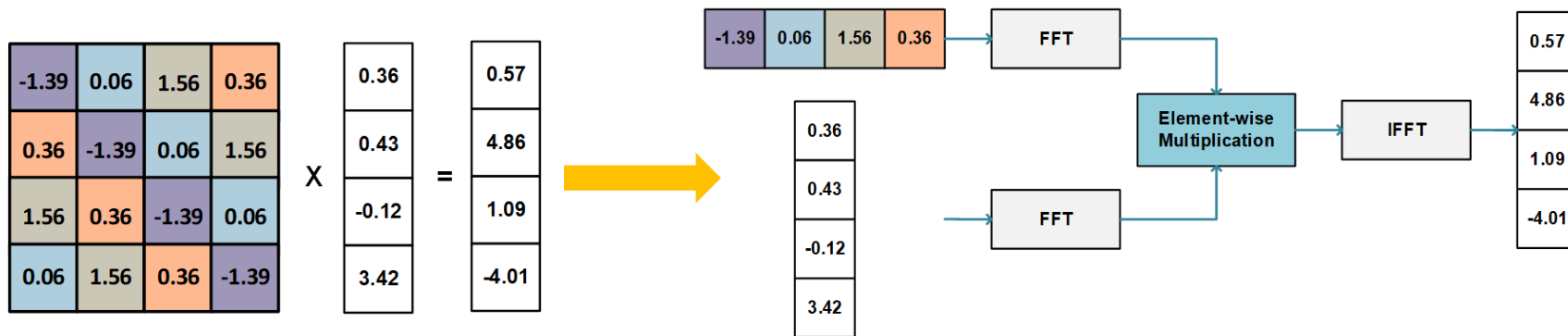
0.36	-1.39	0.06	0.43	-0.24	3.42	-0.12	1.56
-1.39	0.36	0.43	0.06	3.42	-0.24	1.56	-0.12
0.06	1.22	1.72	0.08	1.45	-1.42	0.57	1.47
1.22	0.06	0.08	1.72	-1.42	1.45	1.47	0.57

(b) Proposed

Block-Circulant: achieve a fine-grained tradeoff of accuracy and compression/acceleration.

/ Frequency Domain Sparsity: CirCNN

- FC: Frequency-domain calculation is more efficient



Spatial Matrix Multiplication

Computation: $O(n^2)$

FFT base Frequency-domain Multiplication

Computation: $O(n \log n)$

/ Frequency Domain Sparsity: CirCNN

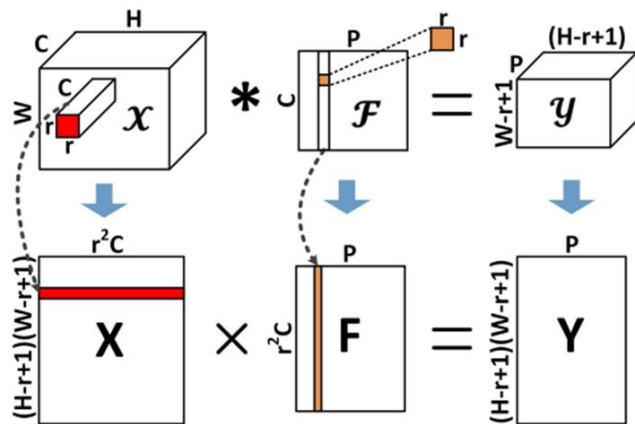


清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

Ding CW@MICRO 2017

- CONV: every “block” (conv kernel) of 4-dimentional weight $F(r, r, C, P)$ is circulant. $C=inc$, $P=outc$

$$y(x, y, p) = \sum_r \sum_r \sum_C \mathcal{F}(i, j, c, p) X(x + i - 1, y + j - 1, c),$$



Computation: $O(WHr^2CP)$

Computation: $O(WHQ\log Q)$, $Q=\max(r^2C, P)$

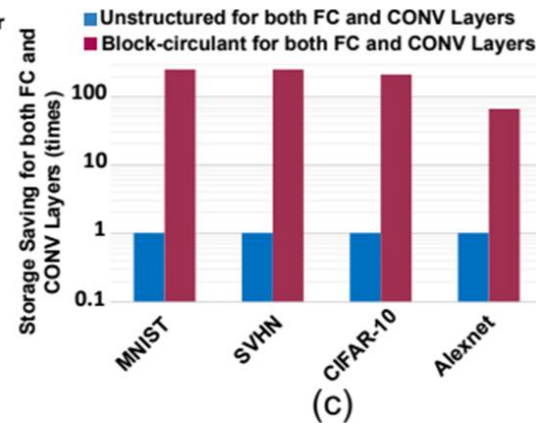
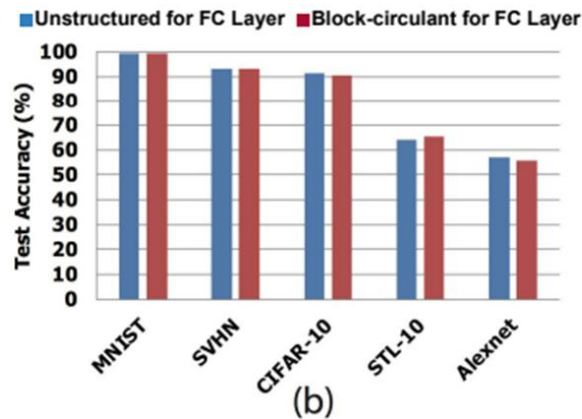
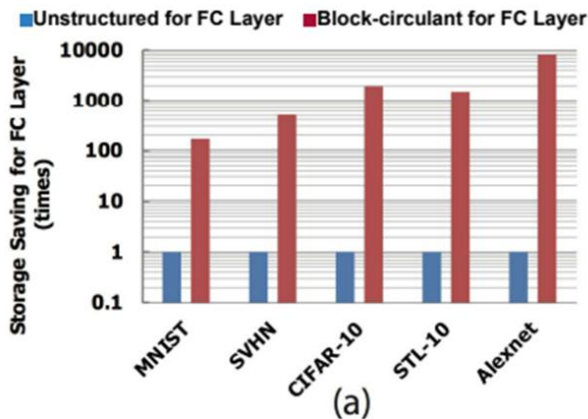
convolution $\rightarrow$ FFT + multiplication + IFFT

/ Frequency Domain Sparsity: CirCNN



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

Ding CW@MICRO 2017

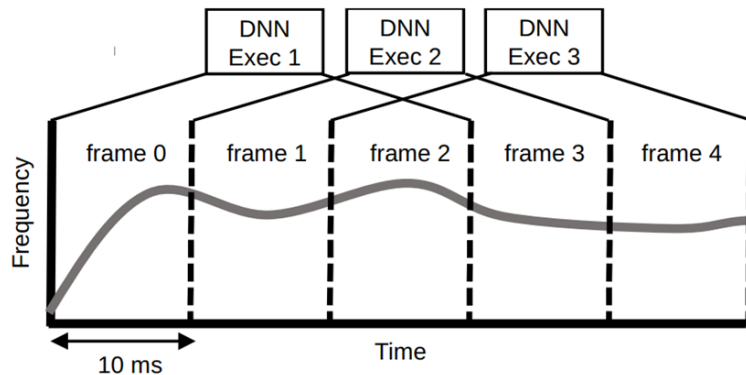


Improvement of CirCNN on MNIST, SVHN, CIFAR-10, STL-10 and AlexNet



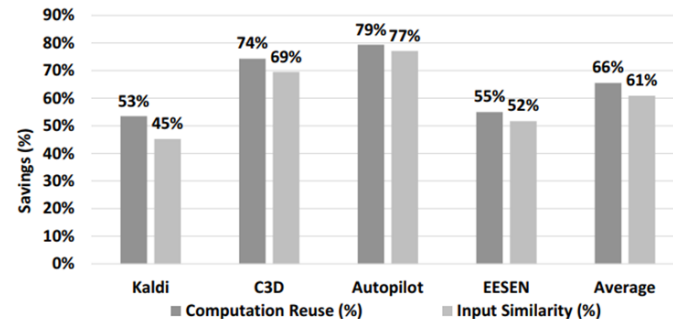
- **Why and How to Achieve Sparse NN Network**
- **Weight Sparsity Perspective**
 - **Unstructured Sparsity**
 - **Structural Sparsity**
 - **Unstructured v.s. Structured: Which is Better?**
 - **Frequency-domain Sparsity**
- **Activation Sparsity Perspective**
 - **Inter-Frame Sparsity**
 - **ROI-Based Sparsity**
- **Leveraging Sparsity in Storage**

/ Inter-Frame Sparsity @Speech Audio

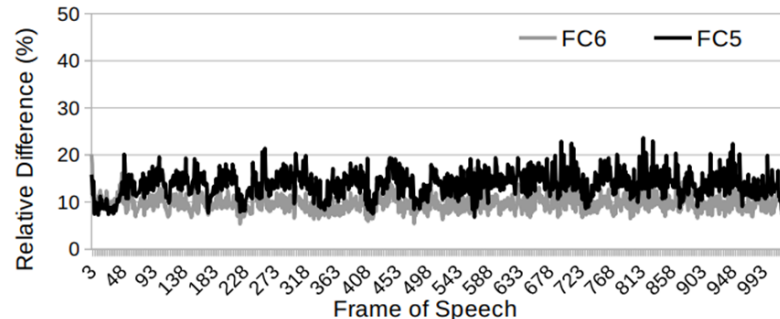


Speech recognition: each DNN is executed as input a windows of three frames. Former results can be used because inter-frame similarity.

Source: [Computation Reuse in DNNs by Exploiting Input Similarity](#)



Input similarity, and compute reuse potentials.



Relative difference (Euclidean distance) between current/previous input vectors.

/ Inter-Frame Sparsity @Speech Audio



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

M. Riera @ISCA 2018

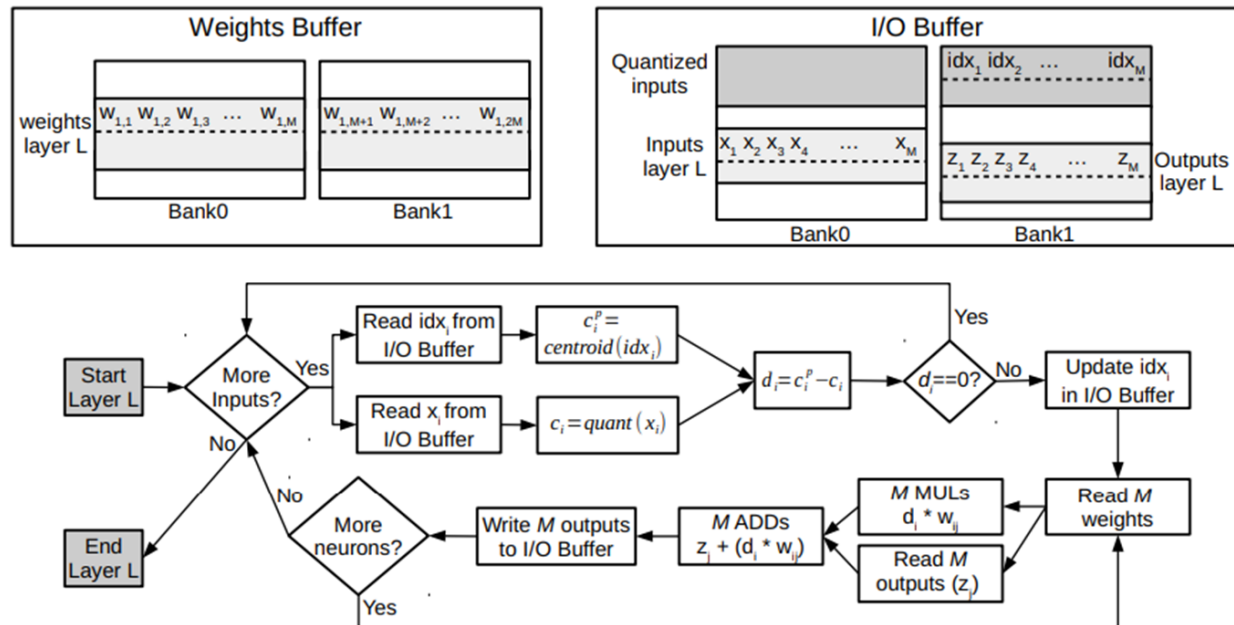
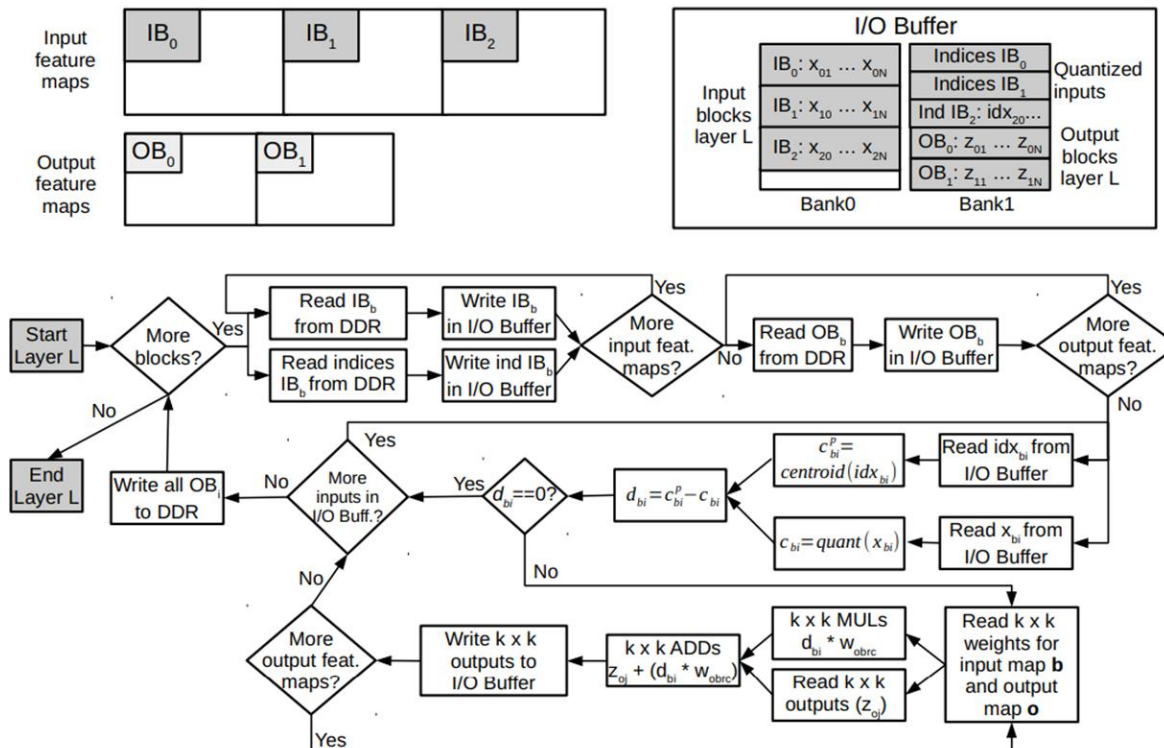


Fig. 7. FC Execution in the Reuse Accelerator.

Intermediate data reuse: FC layers.

/ Inter-Frame Sparsity @Speech Audio



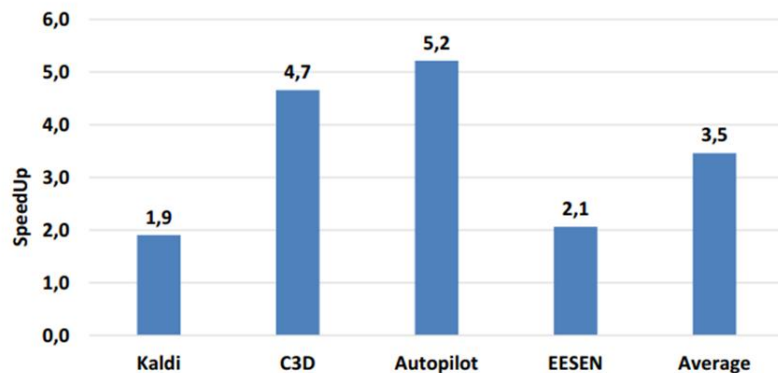
Intermediate data reuse: conv layers.

/ Inter-Frame Sparsity @Speech Audio

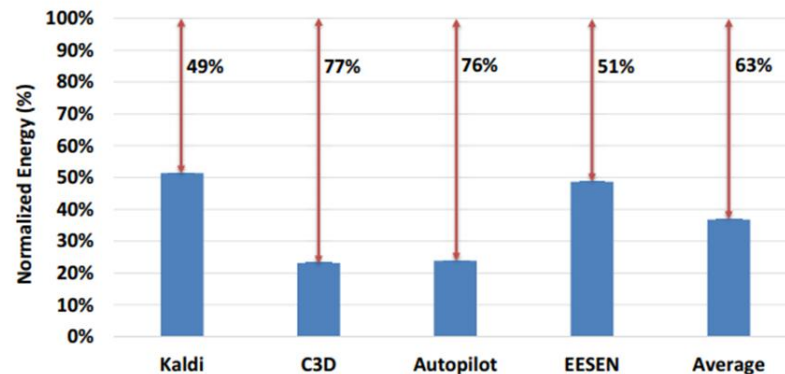


清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

M. Riera @ISCA 2018



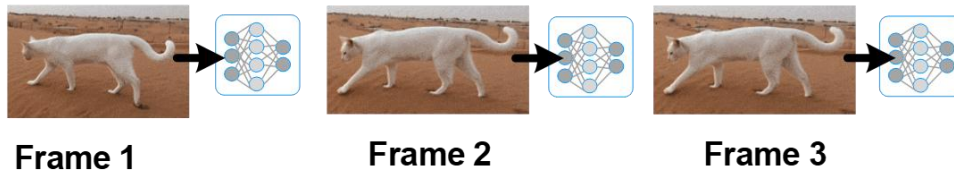
Tested speedups.



Tested energy.

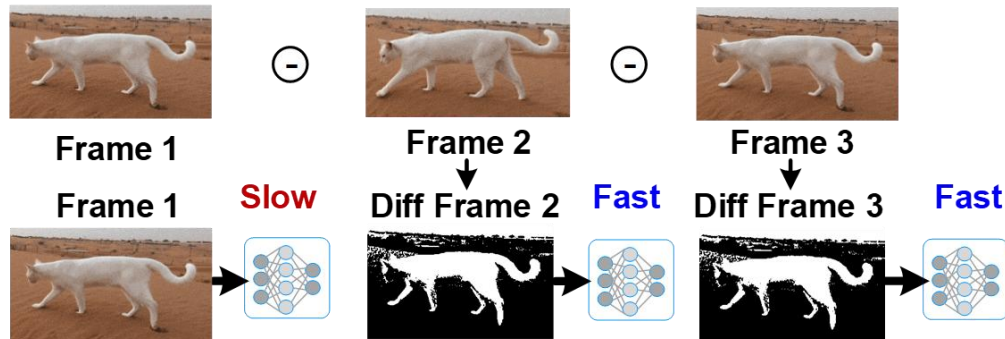
/ Inter-Frame Sparsity @Video

Traditional Way Process One by One



New Dimension Inter Frame Data Reuse

- Faster Processing Time
- Smaller Transmission Data
- Remain Some Challenges

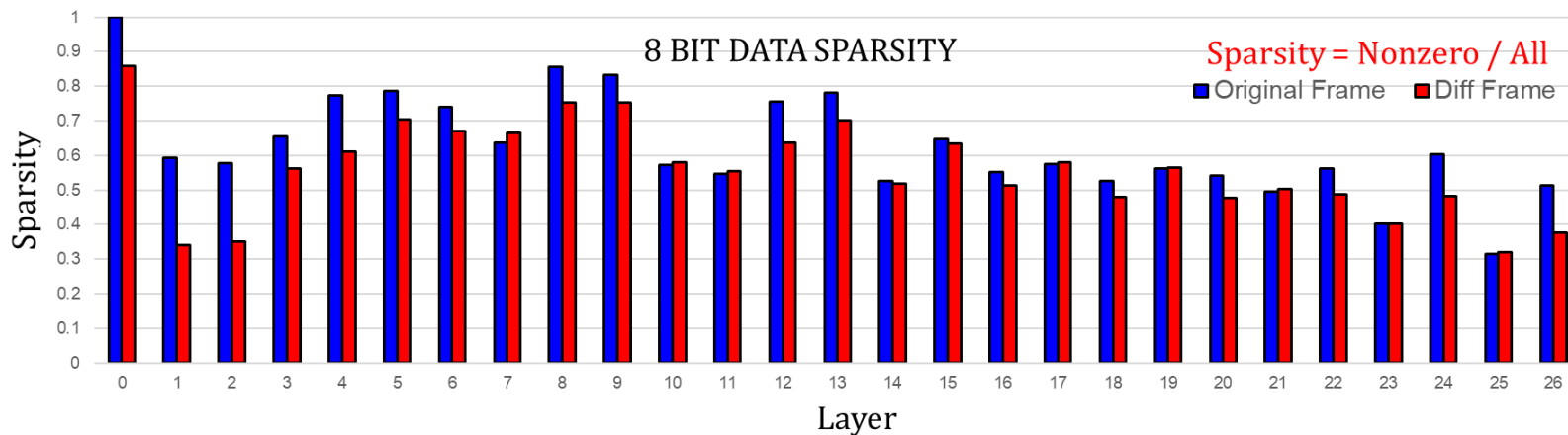


/ Inter-Frame Sparsity @Video

➤ **Observation : Sequential Frames are Similar but Not Identical**

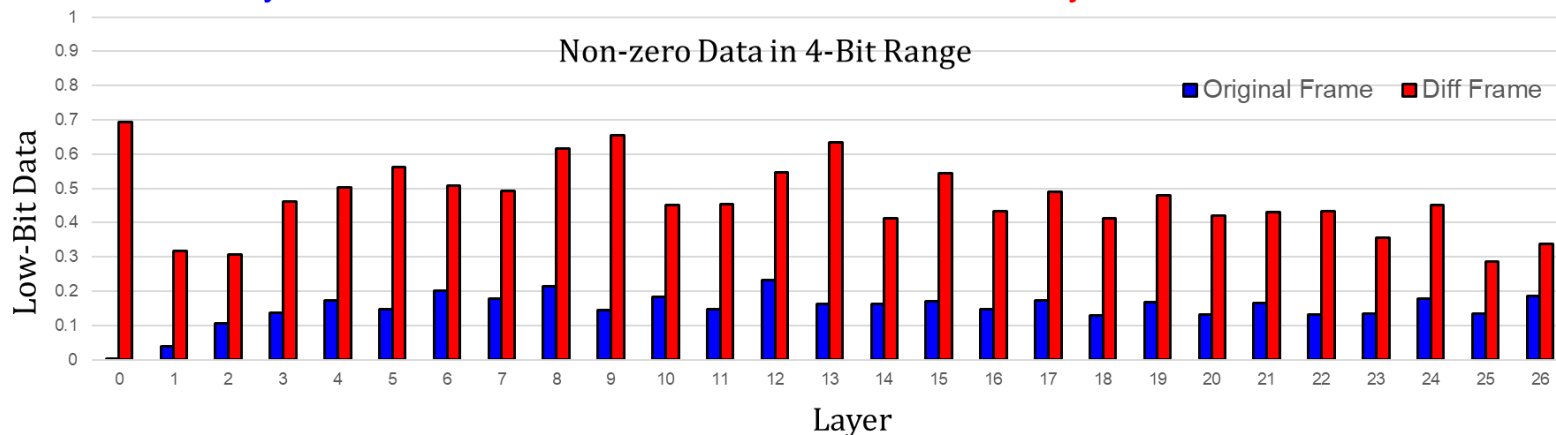
➤ **Viewpoint 1 : Diff Frames Are Not Sparse**

- Only 8.6% sparsity reduced
- Some layers (10, 11, 17) even become more dense
- Inter frame data **cannot** be directly reused



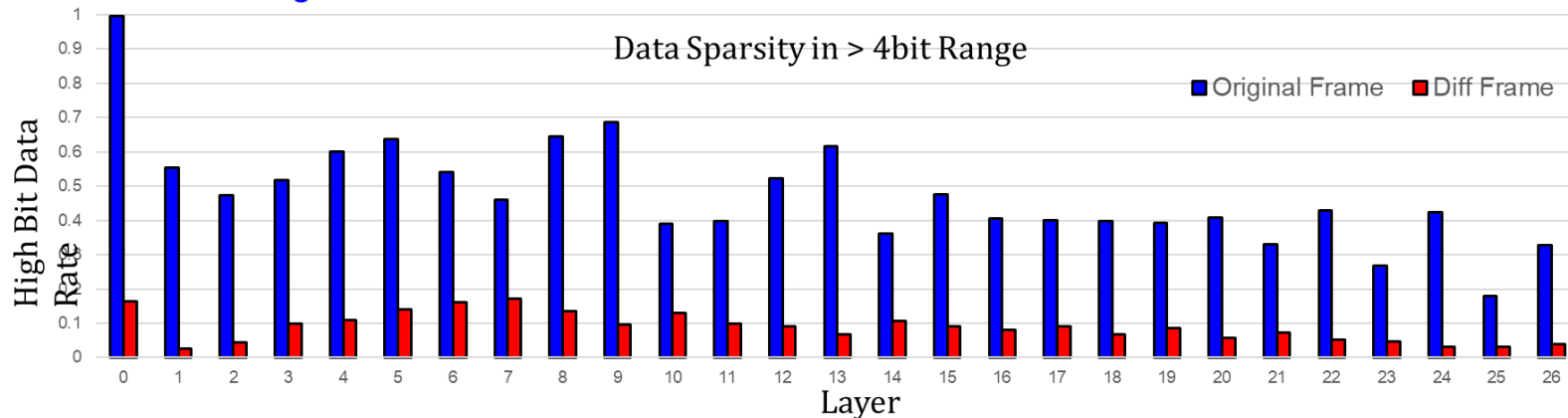
Inter-Frame Sparsity @Video

- **Observation : Sequential Frames are Similar but Not Identical**
- **Viewpoint 1 : Diff Frames Are Not Sparse**
- **Viewpoint 2 : Diff Frames Have More Narrow Distribution**
 - Most of data are reduced to lower bit range
 - Non-zero data in 4-bit range increase by 32.5%
 - Directly discard these low bit data will introduce accuracy loss



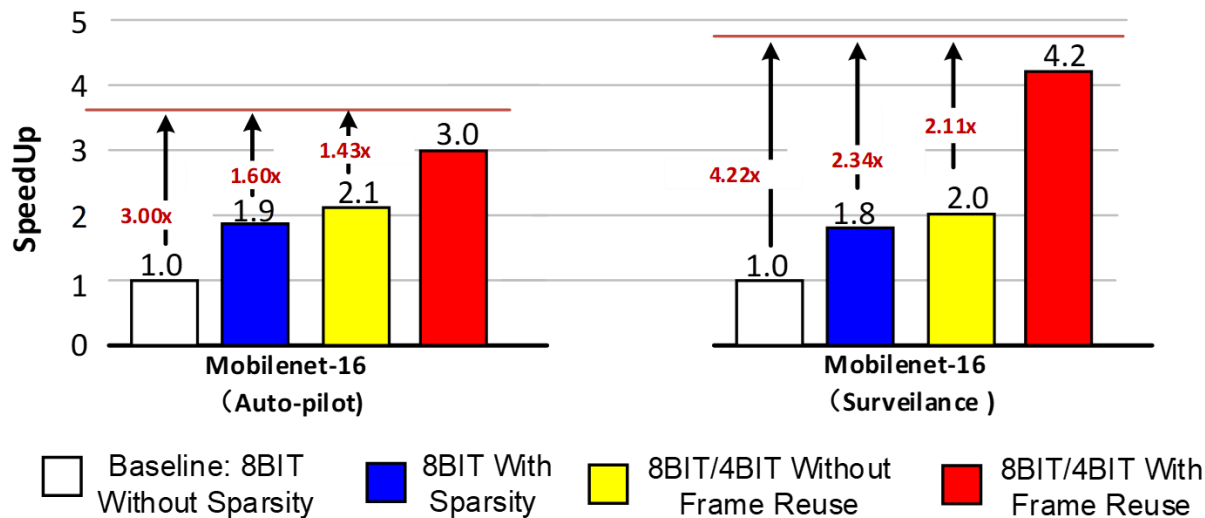
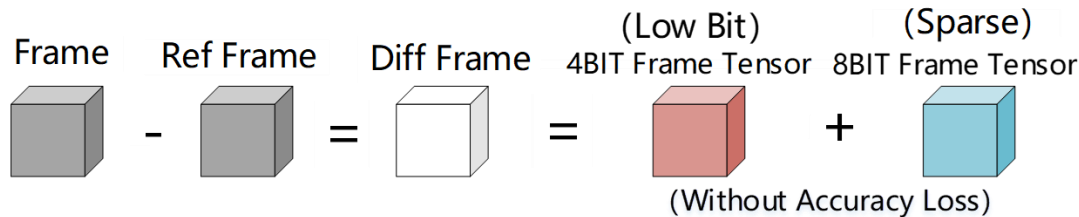
/ Inter-Frame Sparsity @Video

- **Observation : Sequential Frames are Similar but Not Identical**
- **Viewpoint 1 : Diff Frames Are Not Sparse**
- **Viewpoint 2 : Diff Frames have more narrow distribution**
- **Viewpoint 3 : High-bit Data are Very Sparse**
 - Only less than 10% data are > 4 bit
 - These big data still cannot be discarded



/ Inter-Frame Sparsity @Video

Yuan Z @ISSCC 2020





- **Why and How to Achieve Sparse NN Network**
- **Weight Sparsity Perspective**
 - **Unstructured Sparsity**
 - **Structural Sparsity**
 - **Unstructured v.s. Structured: Which is Better?**
 - **Frequency-domain Sparsity**
- **Activation Sparsity Perspective**
 - **Inter-Frame Sparsity**
 - **ROI-Based Sparsity**
- **Leveraging Sparsity in Storage**

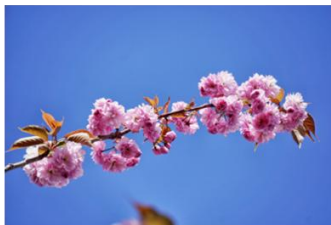
/ ROI Sparsity: Input Dependent



- Images Regions have different information richness



high



medium



low

- Images have different difficulties



easy



difficult

/ ROI Sparsity: Input Dependent

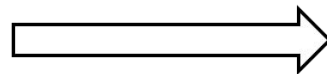
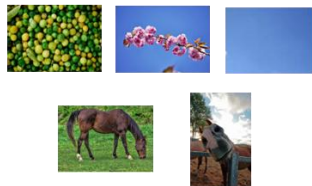


清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

Li XX@CVPR2017

- If using traditional weight pruning method...

- Traditional pruning is region/input independent



Weight Pruned
Model

Different images and regions
have same computing costs!

- How to overcome?

- Consider sparsity in activation



Source: [Not All Pixels Are Equal: Difficulty-Aware Semantic Segmentation via Deep Layer Cascade](#)

/ ROI Sparsity: Input Dependent

Li XX@CVPR2017

	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv	mIoU
FCN [25]	76.8	34.2	68.9	49.4	60.3	75.3	74.7	77.6	21.4	62.5	46.8	71.8	63.9	76.5	73.9	45.2	72.4	37.4	70.9	55.1	62.2
DeepLab [3]	84.4	54.5	81.5	63.6	65.9	85.1	79.1	83.4	30.7	74.1	59.8	79.0	76.1	83.2	80.8	59.7	82.2	50.4	73.1	63.7	71.6
RNN [40]	87.5	39.0	79.7	64.2	68.3	87.6	80.8	84.4	30.4	78.2	60.4	80.5	77.8	83.1	80.6	59.5	82.8	47.8	78.3	67.1	72.0
Adelaide [37]	91.9	48.1	93.4	69.3	75.5	94.2	87.5	92.8	36.7	86.9	65.2	89.1	90.2	86.5	87.2	64.6	90.1	59.7	85.5	72.7	79.1
RNN [†] [40]	90.4	55.3	88.7	68.4	69.8	88.3	82.4	85.1	32.6	78.5	64.4	79.6	81.9	86.4	81.8	58.6	82.4	53.5	77.4	70.1	74.7
BoxSup [†] [6]	89.8	38.0	89.2	68.9	68.0	89.6	83.0	87.7	34.4	83.6	67.1	81.5	83.7	85.2	83.5	58.6	84.9	55.8	81.2	70.7	75.2
DPN [†] [22]	89.0	61.6	87.7	66.8	74.7	91.2	84.3	87.6	36.5	86.3	66.1	84.4	87.8	85.6	85.4	63.6	87.3	61.3	79.4	66.4	77.5
DeepLab-v2 [†] [4]	92.6	60.4	91.6	63.4	76.3	95.0	88.4	92.6	32.7	88.5	67.6	89.6	92.1	87.0	87.4	63.3	88.3	60.0	86.8	74.5	79.7
LC	94.1	63.0	91.2	67.9	79.5	93.4	90.0	93.8	37.4	83.7	65.9	90.7	86.1	88.8	87.5	68.5	86.9	64.3	85.6	72.2	80.3
LC [†]	85.5	66.7	94.5	67.2	84.0	96.1	89.8	93.5	47.2	90.4	71.5	88.9	91.7	89.2	89.1	70.4	89.4	70.7	84.2	79.6	82.7

Per-class segmentation on VOC12 test set

	sub	road	swalk	build.	wall	fence	pole	tlight	sign	veg.	terrain	sky	person	rider	car	truck	bus	train	mbike	bike	mIoU
RNN [40]	2	96.3	73.9	88.2	47.6	41.3	35.2	49.5	59.7	90.6	66.1	93.5	70.4	34.7	90.1	39.2	57.5	55.4	43.9	54.6	62.5
DeepLab [3]	2	97.3	77.7	87.7	43.6	40.5	29.7	44.5	55.4	89.4	67.0	92.7	71.2	49.4	91.4	48.7	56.7	49.1	47.9	58.6	63.1
FCN [25]	no	97.4	78.4	89.2	34.9	44.2	47.4	60.1	65	91.4	69.3	93.9	77.1	51.4	92.6	35.3	48.6	46.5	51.6	66.8	65.3
DPN [22]	no	97.5	78.5	89.5	40.4	45.9	51.1	56.8	65.3	91.5	69.4	94.5	77.5	54.2	92.5	44.5	53.4	49.9	52.1	64.8	66.8
Dilation10 [39]	no	97.6	79.2	89.9	37.3	47.6	53.2	58.6	65.2	91.8	69.4	93.7	78.9	55	93.3	45.5	53.4	47.7	52.2	66	67.1
DeepLab-v2 [4]	no	97.8	81.3	90.3	48.7	47.3	49.5	57.8	67.2	91.8	69.4	94.1	79.8	59.8	93.7	56.5	67.4	57.4	57.6	68.8	70.4
Adelaide [19]	no	98.0	82.6	90.6	44.0	50.7	51.1	65.0	71.7	92.0	72.0	94.1	81.5	61.1	94.3	61.1	65.1	53.8	61.6	70.6	71.6
LC	no	97.9	83.1	91.6	53.7	57.4	58.4	62.0	73.3	91.9	61.3	93.8	78.8	53.1	93.4	62.2	76.9	53.5	57.0	74.7	71.1

Per-class segmentation on Cityspaces test set

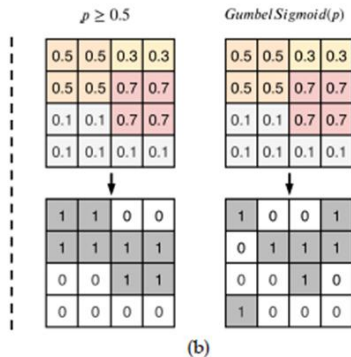
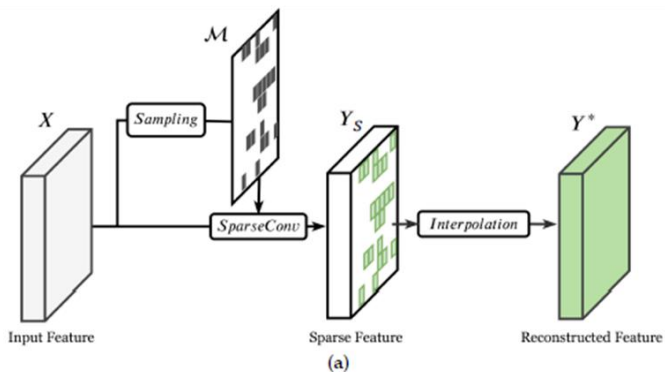
/ ROI Sparsity: Spatially Adaptive Inference



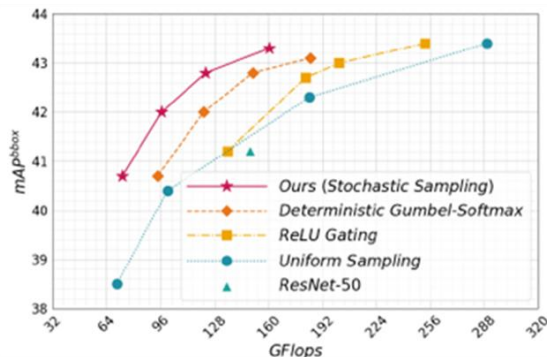
清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

Xie ZD@ECCV2020

- Only the hard pixels go through CNN layers
 - Easy pixels: interpolated by surrounding pixels



Object detection on COCO17 validation



Source: [Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation](#)



- **Why and How to Achieve Sparse NN Network**
- **Weight Sparsity Perspective**
 - **Unstructured Sparsity**
 - **Structural Sparsity**
 - **Unstructured v.s. Structured: Which is Better?**
 - **Frequency-domain Sparsity**
- **Activation Sparsity Perspective**
 - **Inter-Frame Sparsity**
 - **ROI-Based Sparsity**
- **Leveraging Sparsity in Storage**

/ Leveraging Sparsity in Storage



- Goal: reduce memory usage and/or bandwidth
- Commonly-used compression formats:
 - Bitmask (e.g, NVDLA)
 - Run-length encoding (e.g., Eyeriss)
 - Compressed-sparse row (CSR) and compressed sparse column (CSC)
 - Tensor compression (Taco)

/ Bitmask compression

- Using 1-bit/element to indicate whether the value is zero or not.
- Example: NVDLA
- Pros: simple and regular
- Cons: fixed overhead independent of density, e.g., 1/8 overhead for 8-bit values

Original Format

		Column			
Row	A	0	B	C	
	0	0	0	0	
	E	0	0	F	

Compressed Format

Bitmask	1	0	1	1	0	0	0	0	1	0	0	1
Value	A	B	C	E	F							

/ Run-Length Encoding

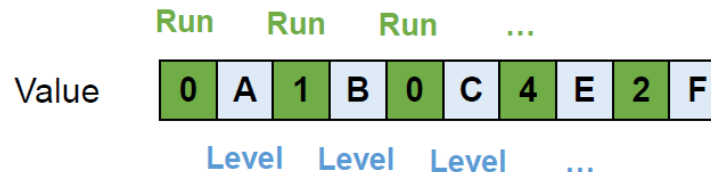


- Runs of data (sequences where the same data value, e.g., 0, occurs in consecutive elements) are stored as a single value and count.
- Example: Eyeriss
- Pros: compression rate linear with nnz (number of non-zero) elements.
- Cons: overhead: $\sim \text{nnz} * (\text{run-bitwidth})$

Original Format

		Column			
Row	A	0	B	C	
	0	0	0	0	
	E	0	0	F	

Compressed Format



/ Compressed Sparse Row (CSR)

- Represents a matrix with 3 one-dimension arrays
 - Bound of each row (size: num of rows)
 - Column index for each nnz element within the row (size: nnz)
 - Non-zero values (size: nnz)
- Pros: allow fast row access

Original Format

		Column		
Row	A	0	B	C
	0	0	0	0
	E	0	0	F

Compressed Format

Bounds (row)	0	3	3	5	
Index (col)	0	2	3	0	3
Value	A	B	C	E	F

/ Compressed Sparse Column (CSC)



- Represents a matrix with 3 one-dimension arrays
 - Bound of each column (size: num of columns)
 - Row index for each nnz element within the column (size: nnz)
 - Non-zero values (size: nnz)
- Pros: allow fast column access

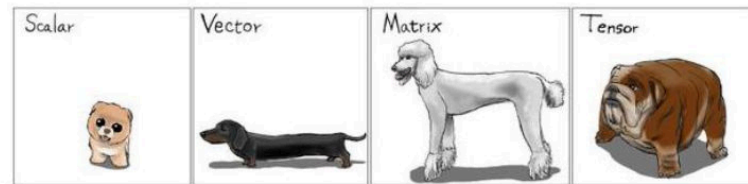
		Column		
Row	A	0	B	C
	0	0	0	0
	E	0	0	F

		Compressed Format				
	Bounds (col)	0	2	2	3	5
	Index (row)	0	2	0	0	2
	Value	A	E	B	C	F

/The Taco Notation



- Tensors: multi-dimensional arrays.
- Tensor storage format:
 - Separately designate each dimension as dense or sparse
 - Specify the order in which dimensions are stored
 - E.g. CSR is a row-dense, column-sparse format.



Anima Anandkumar, ScaledML'2018

	Column			
	A	0	B	C
Row	0	0	0	0
	E	0	0	F

Bounds (row)

0	3	3	5
---	---	---	---

Index (col)

0	2	3	0	3
---	---	---	---	---

Value

A	B	C	E	F
---	---	---	---	---



The Tensor Algebra Compiler, OOPSLA'2017

/The Taco Notation

- Tensors: multi-dimensional arrays.
- Tensor storage format:
 - What if $\text{nnz} < n$?
 - Row-sparse, column-sparse

Original

		Column		
Row	A	0	B	C
	0	0	0	0
	E	0	0	F

CSR

Bounds (row)	0	3	3	5	
Index (col)	0	2	3	0	3
Value	A	B	C	E	F

Sparse-Sparse Doubly compressed sparse row (DCSR)

Bounds (-)	0	2			
Index (row)	0	2			
Bounds (row)	0	3	5		
Index (col)	0	2	3	0	3
Value	A	B	C	E	F



/ Conclusion



- **Why and How to Achieve Sparse NN Network:**
for efficiency, relu + pruning + domain specific
- **Weight Sparsity Perspective**
 - **Unstructured Sparsity: higher sparsity, hard for hardware**
 - **Structural Sparsity: lower sparsity, better for hardware**
 - **Unstructured v.s. Structured: Which is Better?:**
structured better if unstructured PPR < 2.7
 - **Frequency-domain Sparsity:**
circulant, need FFT&IFFT, reduce storage

/ Conclusion

- **Activation Sparsity Perspective**
 - **Inter-Frame Sparsity: audio and video, focusing on residuals**
 - **ROI-Based Sparsity: image, focusing on important pixels**
- **Leveraging Sparsity in Storage: bitmask, run-length, CSR, CSC, Taco**